

Day 04

Service Oriented Architectures

Digital Science Graduate Programme

"SOA is the reason companies like Amazon, Google, and Netflix can have thousands of engineers working in parallel without stepping on each other's toes."

Course Context — Why SOA?

Where We Are

- **Day 04 of Cloud Computing & Big Data Analytics**
- Bridge between distributed systems theory and real-world cloud architecture
- Foundation for everything: ML APIs, data pipelines, cloud deployments

Why SOA Matters To You

1

Every ML model you deploy →
becomes a service

2

Every data pipeline you build →
is an SOA system

3

Every cloud resource you use →
exposes a service API

Lesson 1 — What Is a Service?

OASIS Definition

"A paradigm for organizing and utilizing distributed capabilities that may be under the control of different ownership domains." — OASIS

A Service Is Self-Contained Software That:

- Does one specific job — and does it well
- Exposes that job via a well-defined interface
- Hides internal complexity from callers (abstraction)
- Can be discovered and consumed by other software
- Can be reused by multiple different applications
- Is loosely coupled — internal changes don't break callers

The Vending Machine Mental Model

Vending Machine	SOA Service Equivalent
Button you press	Interface / API Endpoint
Money you insert	Input payload
Coffee you receive	Response / Output
Internal machinery	Implementation (hidden from you)
Menu on the glass	Service Contract (WSDL / OpenAPI)



KEY TAKEAWAY

You don't need to know HOW the coffee is made.

You only need to know which button to press and what you get back.

This is abstraction. This is SOA.

The 7 Core SOA Principles (1/2)

1

Loose Coupling

Minimise dependencies. Interface stays stable even when implementation changes.

2

Service Contract

Every service publishes a formal spec. SOAP → WSDL.
REST → OpenAPI. Never break without versioning.

3

Abstraction

Implementation details hidden from callers. Enables internal upgrades without ripple effects.

4

Reusability

One well-designed service consumed by many applications. Zero code duplication.

The 7 Core SOA Principles (2/2)

5

Composability

Complex apps built from simpler services. Enrolment workflow = grades + finance + timetable.

6

Discoverability

Services published to a registry (UDDI historically, Service Mesh today). Software Yellow Pages.

7

Statelessness

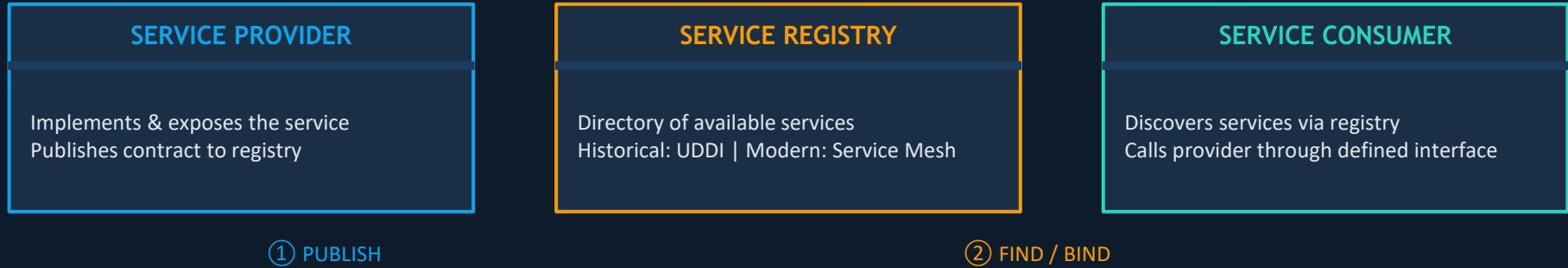
Each call is self-contained. No memory of prior requests. Critical for horizontal scaling.



KEY TAKEAWAY

Every principle solves a specific, painful real-world engineering problem.
They are constraints, not guidelines.

SOA Architecture – The Three Roles



Step	Description
Publish	Provider registers its capabilities with the registry
Find/Bind	Consumer discovers the service via the registry
Call	Consumer calls provider directly after discovery

SOAP vs REST — Protocol Comparison

Dimension	SOAP	REST
Type	Protocol (strict)	Architectural Style
Year	1998 (Microsoft + IBM)	2000 (Roy Fielding PhD)
Message Format	XML only	JSON (typically)
Contract	WSDL — mandatory	OpenAPI — recommended
Transport	HTTP, SMTP, TCP	HTTP / HTTPS
State	Stateful or stateless	Stateless by design
Overhead	High — verbose XML	Low — compact JSON



KEY TAKEAWAY

For the same coffee order: REST produces a payload 8.2× smaller than SOAP.

Build REST. Maintain SOAP. Know the difference.

SOAP — When Enterprise Rules Apply

SOAP Message Structure

<soap:Envelope>

<soap:Header> (security, auth, metadata)

<soap:Body> (actual business payload)

<YourOperation> (request/response)

Use SOAP When:

- Banking transactions and payment gateways
- Healthcare (HL7 FHIR legacy systems)
- Any system where formal contracts + guaranteed delivery are non-negotiable
- Integrating with enterprise legacy systems built in the 2000s

Built-In Enterprise Standards

- WS-Security — message-level encryption & signing
- WS-AtomicTransaction — distributed transactions
- WS-ReliableMessaging — guaranteed delivery

REST — The Modern Default

HTTP Verb	Action	Example Endpoint
GET	Read / Retrieve	GET /students/STU001
POST	Create	POST /grades
PUT	Full Update	PUT /students/STU001
PATCH	Partial Update	PATCH /grades/G42
DELETE	Remove	DELETE /courses/C001

REST Design Rules

- Stateless — every request contains everything needed
- Auth via HTTP header: Authorization: Bearer <JWT>
- Resources identified by URL nouns, not verbs
- Version your APIs: /api/v1/... to avoid breaking changes

The Enterprise Service Bus (ESB)

The Point-to-Point Problem

WITHOUT ESB: 5 services → 10 connections
50 services → 1,225 connections (grows as n^2)

WITH ESB: 50 services → 50 connections
(grows linearly — each service connects only to the bus)

What an ESB Does:

- Routing — directs messages to correct service based on content or rules
- Transformation — converts formats (XML ↔ JSON, SOAP ↔ REST)
- Protocol Mediation — translates between HTTP, JMS, AMQP and others
- Load Balancing — distributes requests across service instances
- Error Handling — manages retries, dead-letter queues, failure alerts

ESB Platforms & Cloud Replacements

Platform	Type	Best For
MuleSoft Anypoint	Commercial	Enterprise integration, API management
IBM App Connect	Commercial	Legacy enterprise, IBM ecosystem
Apache Camel	Open Source	Java integration, rich routing DSL
WSO2 Enterprise Integr.	Open Source	Full SOA stack, open standards

Modern Cloud Replacements (Post-ESB Era)

API Gateway

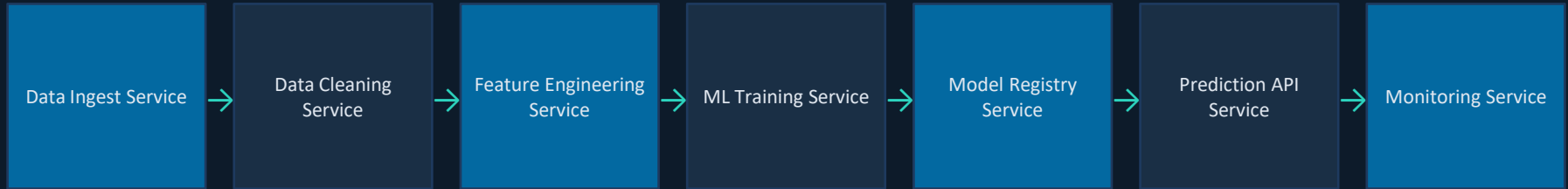
AWS API Gateway, Azure APIM
Handles external clients: auth, rate limits, routing, SSL

Service Mesh

Istio, Linkerd
Handles internal service-to-service traffic in Kubernetes

SOA in Big Data & Cloud Environments

Every Big Data Pipeline Is an SOA System



Cloud-Native SOA Stack

Layer	Technology
Ingestion	Apache Kafka, AWS Kinesis
Orchestration	Apache Airflow, AWS Step Functions
Model Serving	AWS SageMaker, MLflow
API Gateway	AWS API Gateway, Kong

SOA vs Microservices — Cousins, Not Enemies

Dimension	SOA	Microservices
Purpose	Connect disparate enterprise systems	Decompose a single application
Service Size	Coarse-grained (large services)	Fine-grained (tiny, focused)
Communication	ESB-centric (centralised)	Direct HTTP/events (decentralised)
Data Strategy	Often shared databases	Each service owns its own DB
Deployment	Often shared runtime	Independent containers (Docker/K8s)
Technology	Often single platform/language	Polyglot — each service chooses
Primary Use Case	Connect enterprise legacy + modern systems	Build a cloud-native application



KEY TAKEAWAY

Most modern systems use BOTH.

Microservices within each product domain. SOA patterns for cross-domain integration.

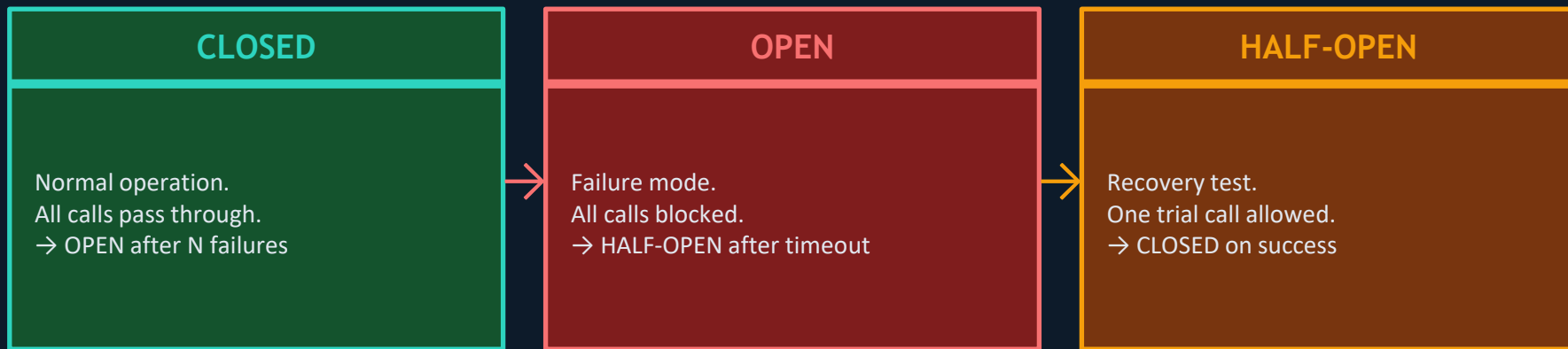
SOA Design Patterns — Overview

Pattern	Problem It Solves	Where You'll See It
Facade	Simplify client access to many services	API Gateways
Aggregator	Combine data from multiple services	Dashboards, portals
Saga	Distributed transactions without locking	E-commerce, banking
Circuit Breaker	Protect against cascading failures	Any resilient SOA system
Event-Driven	Decouple services via asynchronous events	Data pipelines, streaming

Quick Decision Guide

- Need to simplify client access → Facade
- Need to aggregate data sources → Aggregator
- Need distributed transactions → Saga
- Need resilience to failures → Circuit Breaker
- Need real-time streaming pipeline → Event-Driven

Design Pattern – Circuit Breaker

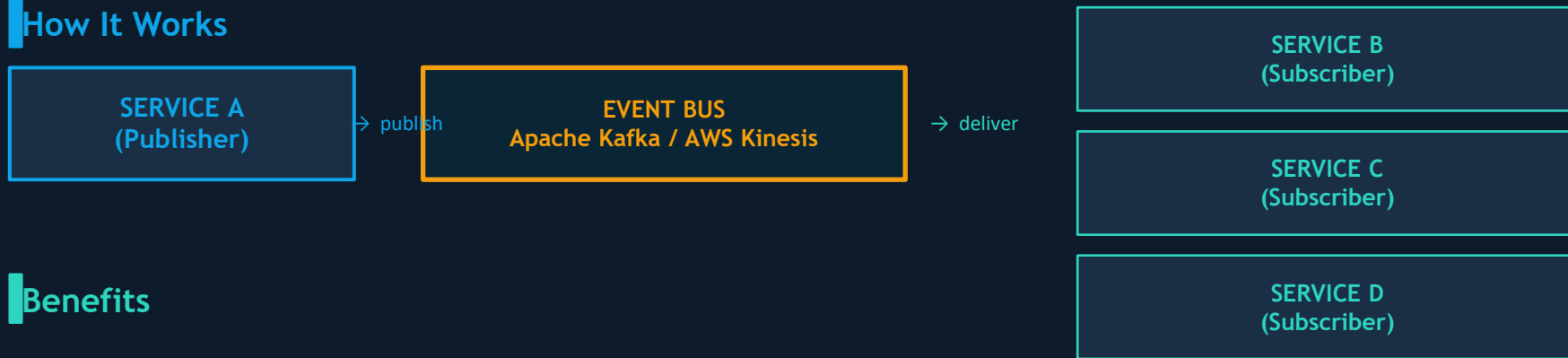


KEY TAKEAWAY

One service failing should NEVER bring down the entire system.
Return an immediate fallback. Retry later. Protect your users.

Design Pattern — Event-Driven SOA

How It Works



Benefits

- Decoupling — publisher has no knowledge of who consumes its events
- Scalability — consumers process at their own pace
- Resilience — events persist; consumers can replay from any point
- Real-time — foundation of all streaming Big Data architectures

Security in SOA – Three Pillars (1/2)

1

Authentication

"Who are you?"

2

Authorization

"What can you do?"

3

Transport Security

"Is it encrypted?"

Authentication Mechanisms

Mechanism	Use Case	Security Level
API Keys	Internal / trusted apps	Low
OAuth 2.0	Delegated auth, public APIs	High
JWT	Stateless auth for REST	High
WS-Security	SOAP / enterprise systems	High

Security in SOA — OWASP Top 10 (2/2)

OWASP API Security Top 10 — Every ML API Deployer Must Know These

- 1 Broken Object Level Authorization
- 2 Broken Authentication
- 3 Excessive Data Exposure
- 4 Lack of Resource Throttling (Rate Limiting)
- 5 Broken Function Level Authorization

- 6 Mass Assignment
- 7 Security Misconfiguration
- 8 Injection
- 9 Improper Asset Management
- 10 Insufficient Logging & Monitoring

Security — Non-Negotiables for Your ML APIs

Checklist: Before You Deploy Any Service

✓ HTTPS

Enforced — never HTTP in production

✓ Auth

Required — minimum API key; better, JWT

✓ Authorisation

Role/scope validation per endpoint

✓ Rate Limiting

Protect against abuse and cost overruns

✓ Input Validation

Never trust incoming data — validate everything

✓ Logging

Every request logged with user ID and timestamp

✓ OWASP Audit

Check Top 10 before launch

Key Abbreviations Reference

Abbreviation	Full Form
SOA	Service Oriented Architecture
ESB	Enterprise Service Bus
API	Application Programming Interface
REST	Representational State Transfer
SOAP	Simple Object Access Protocol
WSDL	Web Services Description Language
UDDI	Universal Description, Discovery and Integration
JSON	JavaScript Object Notation
XML	Extensible Markup Language
JWT	JSON Web Token
TLS	Transport Layer Security
mTLS	Mutual Transport Layer Security
RBAC	Role-Based Access Control
OWASP	Open Web Application Security Project

Master Summary — 6 Things You Must Know

1

Service Definition

Self-contained, interface-defined, implementation-hidden.
Vending machine model.

2

7 SOA Principles

Coupling → Contract → Abstraction → Reuse → Compose →
Discover → Stateless. Constraints, not guidelines.

3

SOAP vs REST

SOAP for enterprise/legacy. REST for everything you build. 8×
more compact.

4

ESB

Central routing/transformation hub. Superseded by API
Gateway + Service Mesh in cloud.

5

SOA vs Microservices

Cousins — both use same principles. SOA connects enterprise
systems. Microservices decompose apps.

6

Design Patterns

Facade · Aggregator · Saga · Circuit Breaker · Event-Driven.
Name it to apply it.

The Career Insight — You're Already Building SOA

What You Build	How SOA Applies
An ML model API	It's a service with a contract and an interface
A data cleaning pipeline	Each step is a composable, loosely coupled service
A recommendation engine	A chain of independent, reusable services
A fraud detection system	An event-driven SOA system on cloud infrastructure
Any cloud deployment	You're registering, discovering, and consuming services



KEY TAKEAWAY

You are already building SOA systems.

Now you know what to call it — and how to do it right.

Resources & Further Reading

Resource	URL
OASIS SOA Standards	https://www.oasis-open.org/
OpenAPI Initiative	https://www.openapis.org/
OWASP API Security	https://owasp.org/www-project-api-security/
Martin Fowler — Microservices	https://martinfowler.com/microservices/
Netflix Tech Blog	https://netflixtechblog.com/
Apache Kafka	https://kafka.apache.org/
Istio Service Mesh	https://istio.io/
JWT Reference	https://jwt.io/
AWS API Gateway	https://aws.amazon.com/api-gateway/

Thank You

Day 04 — Service Oriented Architectures

Cloud Computing & Big Data Analytics | Digital Science Graduate Programme

DISCLAIMER

© Cyber.SoHo — All Rights Reserved. Intellectual Property of Cyber.SoHo.
Educational use only. No affiliation with any listed organisation, platform, or product is implied.
All trademarks are property of their respective owners.
Verify all external links via <https://www.virustotal.com/gui/home/url> before accessing.
Code examples are for educational demonstration only — not production-ready.